

CIS350 Winter 2026: Assignment 1

- Implementing/Extending a Doubly Linked List &
- Implementing/Extending a Score Management System

Objective:

In this assignment, you will extend the implementation of a Doubly Linked List (DLL) and implement additional functions for a score management system using Arrays, Vectors, or Lists.

You will demonstrate your understanding of linked data structures, dynamic arrays, and algorithm design by implementing, testing, and analyzing the behavior of your extended Doubly Linked List and the Score Management System. Choose only 1 programming language from:

C++: See section 3.1 for Score Management System and section 3.3 for Doubly Linked Lists

Java: See section 3.1 for Score Management System and section 3.4 for Doubly Linked Lists

Python: See section 5.5 for Score Management System and section 7.3 for Doubly Linked Lists

Part 1: Doubly Linked List

1. Starting Code:

- You are provided with a basic implementation of a Doubly Linked List (DLinkedList), which includes basic functions such as adding and removing elements from the front and back and printing the list. Study the provided code carefully to understand its structure and functionality. **Note that the given addBefore() method adds an element before the given node. You should be able to demonstrate in testing how an element can be added after an existing element other than the front and back of the list.**

Note: Your submission will contain same name and number of files as given. Only provided code content will change.

- The starting code will differ slightly based on the language you choose. Modify as appropriate:
 - C++: Header files and implementation files.
 - Java: Classes with constructors and methods.
 - Python: Class definitions with methods.

2. Tasks:

You need to implement the following additional functions in the DLinkedList class:

1. Implement the Find Function:

- Function Signature:
 - a. C++: `int find(const Ele& element);`
 - b. Java: `int find(String element);`
 - c. Python: `def find(self, element):`
- **Description:** Find the first occurrence of the specified element (element) in the list. Return an integer indicating the position of the element found in the list containing the element if found, otherwise return -1. If the element is found, print a message indicating its position in the list.

2. Insert After a Given Element:

- Function Signature:
 - a. C++: `void insertAfter(const Ele& existingElement, const Ele& newElement);`
 - b. Java: `public void insertAfter(DNode existingElement, String newElement);`
 - c. Python: `def insert_after(self, existing_element, new_element):`
- **Description:** Insert a new element (newElement) after a given element (existingElement). If the existingElement is not found, print a suitable message.

3. Reverse the List:

- Function Signature:
 - a. C++: `void reverse();`
 - b. Java: `void reverse();`
 - c. Python: `def reverse(self):`
- **Description:** Reverse the order of elements in the list. This operation should modify the list in place so that the last element now becomes the first element, second last becomes the second element and so on in the modified list. When printed by the `printList()` function it print the list in reverse order.

4. Add an element at Back

- Function Signature:
 - a. C++: `void addBack(const Ele& e);`
 - b. Java: `void addBack(String newElement);`
 - c. Python: `def addBack(self):`
- **Description:** add a new element (newElement) at the back of the list).

5. Remove an element from Back

- Function Signature:
 - a. C++: `void removeBack();`
 - b. Java: `void removeBack();`
 - c. Python: `def removeBack(self):`
- **Description:** remove the element from the back of the list.

3. Testing and Output:

C++ Test Plan

A good testing strategy checks for various scenarios to ensure your code is robust. This plan follows logical progression:

4. **Initial State & Edge Cases:** Start by testing the list when it's empty. This checks if your functions can handle the absence of nodes gracefully.

5. **Populating the List:** Add elements using addFront and the new addBack function to build a list.
6. **Core Functionality:**
 - Test find() with elements at the beginning, middle, end, and one that doesn't exist.
 - Test insertAfter() in the middle of the list and for a non-existent element.
7. **Manipulation:** Use reverse() to modify the list in place and verify the new order.
8. **Depopulating the List:** Remove elements using removeFront and removeBack until the list is empty again, checking its state along the way.
 - Use the provided DLinkedListTest.cpp file to test all the newly implemented functions.
 - It incorporates the test plan above and is designed to test all the functions mentioned in the assignment.

C++ Expected Output

Use the provided test program, DLinkedListTest.cpp , to test the functions you implemented:

These test functions should produce the following output on your screen. You can use this to verify your implementation.

C++ Output:

```
--- 1. Testing on an empty list ---
Is the list empty? True
List contents: List is empty
Attempting to remove from an empty list...

--- 2. Populating the list ---
Adding elements: SFO, PVD (front) and JFK, BOS (back)
Current list: SFO -> PVD -> JFK -> BOS
Front element: SFO
Back element: BOS
Is the list empty? False

--- 3. Testing find() function ---
Element 'PVD' found at position 2
Element 'SFO' found at position 1
Element 'BOS' found at position 4
Element 'MIA' not found in the list.

--- 4. Testing insertAfter() function ---
Inserting 'BWI' after 'PVD'...
List after insertion: SFO -> PVD -> BWI -> JFK -> BOS
Attempting to insert 'MIA' after non-existent 'LAX'...
Element 'LAX' not found. Cannot insert after it.
List remains: SFO -> PVD -> BWI -> JFK -> BOS

--- 5. Testing reverse() function ---
Reversing the list...
List after reverse: BOS -> JFK -> BWI -> PVD -> SFO
New front element: BOS
New back element: SFO

--- 6. Testing removeFront() and removeBack() ---
Removing from front...
List after removeFront: JFK -> BWI -> PVD -> SFO
Removing from back...
List after removeBack: JFK -> BWI -> PVD

--- 7. Emptying the list ---
List after removing all elements: List is empty
```

Is the list empty? True

Java Test Plan:

The testing strategy for the Java implementation mirrors the logic for the C++ version to ensure all requirements are met. The flow is as follows:

1. **Empty List Tests:** Begin by verifying that all functions behave correctly on a newly created, empty list.
2. **List Population:** Use a combination of `addFront` and the new `addBack` method to build a sample list.
3. **Core Functionality Tests:**
 - `find()`: Test the `find()` method by searching for elements at the beginning, middle, and end of the list, as well as an element that does not exist. The function should return a 1-based index or -1.
 - `insertAfter()`: Test the insertion method.

Note: The Java signature is `insertAfter(DNode existingElement, String newElement)`. To use this effectively, your `DLinkedList` class will need a helper method like `public DNode findNode(String element)` that returns the node object. The test code below assumes you've created this helper.

4. **List Manipulation:** Call the `reverse()` method to alter the list in-place and then print it to confirm the new order.
5. **List Depopulation:** Use `removeFront` and `removeBack` to take elements off both ends of the list, verifying the list's state after each removal.

Expected Java Output

Use the provided test program, DLinkedListTest.java, to test the functions you implemented:

These test functions should produce the following output on your screen. You can use this to verify your implementation.

Java Output:

If your Java methods are implemented correctly, running the test class above will produce this exact output.

--- 1. Testing on an empty list ---

Is the list empty: True

List contents:

List is empty

Attempting to remove from an empty list...

--- 2. Populating the list ---

Adding elements: SFO, PVD (front) and JFK, BOS (back)

Current list:

SFO

PVD

JFK

BOS

Front element: SFO

Back element: BOS

Is the list empty: False

--- 3. Testing find() function ---

Element 'PVD' found at position 2

Element 'SFO' found at position 1

Element 'BOS' found at position 4

Element 'MIA' not found in the list.

--- 4. Testing insertAfter() function ---

Inserting 'BWI' after 'PVD'...

List after insertion:

SFO

PVD

BWI

JFK

BOS

Attempting to insert 'MIA' after non-existent 'LAX'...

Element 'LAX' not found. Cannot insert after it.

List remains:

SFO
PVD
BWI
JFK
BOS

--- 5. Testing reverse() function ---

Reversing the list...

List after reverse:

BOS
JFK
BWI
PVD
SFO

New front element: BOS

New back element: SFO

--- 6. Testing removeFront() and removeBack() ---

Removing from front...

List after removeFront:

JFK
BWI
PVD
SFO

Removing from back...

List after removeBack:

JFK
BWI
PVD

--- 7. Emptying the list ---

List after removing all elements:

List is empty

Is the list empty: True

Python Test Plan

To ensure your Python implementation is robust, the test code will follow a logical sequence that covers all requirements from the assignment description.

1. **Empty List Tests:** The script first checks the behavior of your list methods when the list is empty.
2. **List Population:** Elements are added to both the front and back of the list using `addFront` and the new `addBack` method.
3. **Core Functionality Tests:**
 - **find():** The script searches for elements at various positions (start, middle, end) and also for an element that is not in the list to test the "not found" case. The function should return a 1-based index.
 - **insert_after():** An element is inserted after an existing element to test the primary functionality. It also attempts to insert after a non-existent element to verify error handling.
4. **List Manipulation (reverse()):** The `reverse()` method is called to modify the list in-place, and the result is printed to confirm the order has been successfully reversed.
5. **List Depopulation (removeFront, removeBack):** Elements are removed from both ends of the list to ensure the `removeFront` and `removeBack` methods work correctly on a populated list.

Expected Python Output

Use the provided Python program, `chap7-doubly-linked-list.py`, to test the functions you implemented:

These test functions should produce the following output on your screen. You can use this to verify your implementation.

Python Output:

If your Python methods are implemented correctly, running the test class above will produce this exact output.

```
--- 1. Testing on an empty list ---
Is the list empty: True
List contents:
List is empty
Attempting to remove from an empty list...

--- 2. Populating the list ---
Adding elements: SFO, PVD (front) and JFK, BOS (back)
Current list:
SFO
PVD
JFK
BOS
Front element: SFO
```

Back element: BOS
Is the list empty: False

--- 3. Testing find() function ---

Element 'PVD' found at position 2

Element 'SFO' found at position 1

Element 'BOS' found at position 4

Element 'MIA' not found in the list.

--- 4. Testing insert_after() function ---

Inserting 'BWI' after 'PVD'...

List after insertion:

SFO

PVD

BWI

JFK

BOS

Attempting to insert 'MIA' after non-existent 'LAX'...

Element 'LAX' not found. Cannot insert after it.

List remains:

SFO

PVD

BWI

JFK

BOS

--- 5. Testing reverse() function ---

Reversing the list...

List after reverse:

BOS

JFK

BWI

PVD

SFO

New front element: BOS

New back element: SFO

--- 6. Testing removeFront() and removeBack() ---

Removing from front...

List after removeFront:

JFK

BWI

PVD

SFO

Removing from back...

List after removeBack:

JFK
BWI
PVD

--- 7. Emptying the list ---

List after removing all elements:

List is empty

Is the list empty: True

Part 2: Score Management System

1. Starting Code:

- You are provided with basic classes, GameEntry and Scores that manage a list of player scores using Arrays, Vectors, or Lists.
- Study the provided code carefully to understand its structure and functionality.
- The starting code will differ slightly based on the language you choose. Modify as appropriate:
 - C++: Classes with std::vector for dynamic arrays.
 - Java: Classes with ArrayList.
 - Python: Classes with native list.

2. Tasks:

You need to implement the following additional functions in the Scores class:

1. Search the Score of a Particular Player:

- Function Signature:
 - C++: `int searchScore(const std::string& playerName) const;`
 - Java: `int searchScore(String playerName);`
 - Python: `def search_score(self, player_name):`
- Description: Search the score of a player by name. Return the score if found else return -1.

2. Calculate the Average of the Maximum Scores:

- Function Signature:
 - C++: `double averageMaxScores() const;`
 - Java: `double averageMaxScores();`
 - Python: `def average_max_scores(self):`
- Description: Calculate and return the average of all scores stored in the Scores list.

3. Find the Minimum and Maximum Scores:

- Function Signature:
 - C++: `std::pair<int, int> findMinMaxScores() const;`
 - Java: `int[] findMinMaxScores();`
 - Python: `def find_min_max_scores(self):`
- Description: Find and return the minimum and maximum scores as a pair of integers.

3. Testing and Output:

This section outlines the test plans and expected outputs for the Score Management System implementation in C++, Java, and Python.

Common Task Areas

Across C++, Java, and Python, the core logic for testing the new Scores class functionalities remain the same. A comprehensive test plan for any of these languages should include the following steps:

1. Setup and Population:

- Instantiate a Scores object.
- Create several GameEntry objects with different names and scores.
- Add these entries to the Scores object.

2. Testing Core Functionality (on a populated list):

- **Search:** Test the `searchScore` function for a player who exists in the list and for a player who does not.
- **Average:** Test the `averageMaxScores` function to ensure it correctly calculates the average of all scores.
- **Min/Max:** Test the `findMinMaxScores` function to verify it correctly identifies the lowest and highest scores in the list.

3. Testing Edge Cases (on an empty list):

- Instantiate a *new, empty* Scores object.
- Call `searchScore`, `averageMaxScores`, and `findMinMaxScores` on this empty object. This is crucial to demonstrate that your code handles these situations gracefully (e.g., doesn't crash, returns sensible default values like -1, 0.0, or 0s for min/max).

4. Output and Verification:

- Print clear, descriptive messages before and after each test.

- Display the results from each function call to demonstrate correctness. This matches the sample outputs and testing requirements.
-

C++ Implementation

C++ Test Plan

The C++ test plan will use

`std::vector` to manage `GameEntry` objects within `Scores` class. The

`findMinMaxScores` function is specifically required to return a `std::pair<int, int>`. The test code will be placed in `GameEntryTest.cpp`.

1. Create a `Scores` object named `BAM` and populate it with the provided `GameEntry` objects.
2. Call `BAM.searchScore("Rob")` to find an existing player's score.
3. Call `BAM.searchScore("Zelda")` to test the case where a player is not found.
4. Call `BAM.averageMaxScores()` and print the result.
5. Call `BAM.findMinMaxScores()`, store the returned `std::pair`, and print the `.first` (min) and `.second` (max) values.
6. Create a second, empty `Scores` object `emptyScores`.
7. Repeat steps 2-5 on `emptyScores` to verify edge case handling.

Expected C++ Output

--- Populating scores ---

Initial list of scores:

Mike : 1105

Rob : 750

Paul : 720

Anna : 660

Rose : 590

Jack : 510

Jill : 740

--- 1. Testing on a populated list ---

Searching for Rob's score...

Rob's score is: 750

Searching for Zelda's score...

Zelda not found.

Calculating the average score...

Average score: 725

Finding minimum and maximum scores...

Minimum score: 510

Maximum score: 1105

--- 2. Testing on an empty list ---

Searching for player in empty list...

Mario not found.

Average of empty list: 0

Min score in empty list: 0

Max score in empty list: 0

Java Implementation

Java Test Plan

The Java test plan uses an ArrayList to manage GameEntry objects. The

findMinMaxScores function is required to return a primitive int[] array of size 2, where the first element is the minimum and the second is the maximum. The test code will be placed in

TestGameScoreManagement.java.

1. Create a Scores object BAM and add the sample GameEntry objects.
2. Test BAM.searchScore("Rob") and BAM.searchScore("Zelda").
3. Test BAM.averageMaxScores().
4. Test BAM.findMinMaxScores(), storing the result in an int[] and printing the values at index 0 (min) and 1 (max).
5. Create a new, empty Scores object emptyScores.
6. Repeat the tests on emptyScores to validate edge case behavior.

Expected Java Output

--- Populating scores ---

Initial list of scores:

Mike : 1105

Rob : 750

Paul : 720

Anna : 660

Rose : 590

Jack : 510

Jill : 740

--- 1. Testing on a populated list ---

Searching for Rob's score...
Rob's score is: 750
Searching for Zelda's score...
Zelda not found.
Calculating the average score...
Average score: 725.0
Finding minimum and maximum scores...
Minimum score: 510
Maximum score: 1105

--- 2. Testing on an empty list ---
Searching for player in empty list...
Mario not found.
Average of empty list: 0.0
Min score in empty list: 0
Max score in empty list: 0

Python Implementation

Python Test Plan

The Python test plan will use a native Python list to manage GameEntry objects. The find_min_max_scores function should return a tuple (min_score, max_score)¹⁵. The test code will be placed inside the

if __name__ == "__main__": block of the chap5-score-mgmt.py file.

1. Instantiate a Scores object BAM and add the GameEntry objects.
2. Test BAM.search_score("Rob") and BAM.search_score("Zelda").
3. Test BAM.average_max_scores().
4. Test BAM.find_min_max_scores(), using tuple unpacking to receive the min and max values.
5. Create a new, empty Scores object empty_scores.
6. Repeat the tests on empty_scores to check edge case handling.

Expected Python Output

--- Populating scores ---
Initial list of scores:
Mike : 1105
Rob : 750
Paul : 720
Anna : 660

Rose : 590

Jack : 510

Jill : 740

--- 1. Testing on a populated list ---

Searching for Rob's score...

Rob's score is: 750

Searching for Zelda's score...

Zelda not found.

Calculating the average score...

Average score: 725.0

Finding minimum and maximum scores...

Minimum score: 510

Maximum score: 1105

--- 2. Testing on an empty list ---

Searching for player in empty list...

Mario not found.

Average of empty list: 0.0

Min score in empty list: None

Max score in empty list: None

(Note: The min/max for an empty Python list could reasonably be 0 or None. Returning None is often preferred to distinguish from a list containing the score 0. The exact output here will depend on your specific implementation.)

Submission Requirements:

Submit the following files for both parts:

3. C++: .h and .cpp files containing class declarations and implementations for both parts.
4. Java: .java files containing class implementations for both parts.
5. Python: .py files containing class implementations and test cases for both parts.

Ensure that your code compiles (for C++ and Java) or runs without errors (for Python).

Grading Criteria:

- Ensure your code is well-organized and properly commented. Use meaningful variable names and follow good coding practices.
- Correctness: The functions should work as specified and handle edge cases (e.g., player not found, empty list).

- Code Quality: Code should be well-structured, properly commented, and follow best practices.
- Testing: The main() function should test the newly implemented functions with a variety of inputs.

Submission Guidelines

Screenshots of Outputs:

Include screenshots of your program outputs in a separate MS Word document to demonstrate the functionality of your code. Show the testing of all the existing and new functions.

Doubly Linked List Program (DLinkedList.h and DLinkedList.cpp):

- You are provided with 5 function signatures in C++, Java and Python

Note: the 'findNode' function is used to locate and return the node corresponding to a given element and will be passed as the first parameter in the following function:

void insertAfter(const Ele& existingElement, const Ele& newElement);

Note: int find(const Ele& element);

This function should return the position of the given element in the list as an integer. The first element in the list should be considered to be at position 1, the second at position 2, and so on.

Testing the Programs:

- For both programs (Doubly Linked List and Score Management), keep the original main() function and modify it as needed for testing purposes. Do not create a new main() function.
- For Python implementation:
Include all your code in a single file.

Score Management Program:

- **Note:**
 - C++ Users: For the findMinMaxScores() function, it should return a std::pair<int, int> containing the minimum score (as the first value) and the maximum score (as the second value). The pair data structure is part of the C++ Standard Library (std).
 - Python Users:
Python also has a similar data structure, so ensure your implementation uses the equivalent structure to return the minimum and maximum scores.

Submission Guidelines:

1. Organize your files into folders as follows:

- Doubly Linked List Program: Place all related files along with the screenshots in MS Word file in a folder named 'DLinkedListProgram'.
- Score Management Program: Place all related files along with the screenshots in MS Word file in a folder named 'ScoreManagementProgram'.

2. Create a separate submission folder:

- **The name of this folder should follow the format:**
XXX_Assignment1_YourName_CIS350,
where:

1. **XXX** represents the programming language you have chosen.

2. **YourName** should be replaced with your first and last name.

For example:

- If you are submitting the assignment in C++, name the folder: **CPP_Assignment1_YourName_CIS350**.
- If you are submitting the assignment in Python, name the folder: **Python_Assignment1_YourName_CIS350**.

3. Bundling and Submitting:

- **Copy and paste the 2 program folders (DLinkedListProgram, and ScoreManagementProgram) under this folder (XXX_Assignment1_YourName_CIS350)**
 - **Enzip this folder into zip file name, XXX_Assignment1_YourName_CIS350.zip**
 - **Make sure all your code and screenshots are properly organized into the respective folders before submission.**
 - **Correct bundling and submission format will carry 5 points out of 100.**
-

Please follow the submission guidelines for Assignment 1 based on the programming language you have chosen for your implementation. Ensure you submit the appropriate files for both the Doubly Linked List and Score Management System programs as detailed below.

For C++:

You are required to submit 8 files in total:

1. Doubly Linked List Program (3 files):

- **DLinkedList.h**: This file should contain the class declarations.
- **DLinkedList.cpp**: This file should contain the class implementations.
- **DLinkedListTest.cpp**: This file should include the testing of all functions implemented. (*Refer to the attached file for a sample output.*)

2. Score Management Program (5 files):

- **GameEntry.h**: This file should contain the class declarations for the GameEntry class.
- **Scores.h**: This file should contain the class declarations for the Scores class.
- **GameEntry.cpp**: This file should contain the class implementations for the GameEntry class.
- **Scores.cpp**: This file should contain the class implementations for the Scores class.
- **GameEntryTest.cpp**: This file should include the testing of all functions implemented. (*Refer to the attached file for a sample output.*)

For Java:

You are required to submit 5 files in total:

1. Doubly Linked List Program (2 files):

- **DLinkedList.java**: This file should contain both the class declarations and implementations.
- **DLinkedListTest.java**: This file should include the testing of all functions implemented. (*Refer to the attached file for a sample output.*)

2. Score Management Program (3 files):

- **GameEntry.java:** This file should contain the class declarations for the GameEntry class.
- **Scores.java:** This file should contain the class declarations for the Scores class.
- **TestGameScoreManagement.java:** This file should include the testing of all functions implemented. *(Refer to the attached file for a sample output.)*

For Python:

You are required to submit 2 files in total:

1. Doubly Linked List Program (1 file):

- **chap7-doubly-linked-list.py:** This file should contain class declarations, implementations, and testing of all functions.

2. Score Management Program (1 file):

- **chap5-score-mgmt.py:** This file should contain class declarations, implementations, and testing of all functions.

Additional Requirement:

Along with the code files, please also submit screenshots showing the output of your 2 programs for each function you have implemented. These screenshots should demonstrate the working of your programs. *(Refer to the attached file for a sample output.)*

C++ Assignment Rubric: Doubly Linked List & Score Management Programs

Total Points: 100

1. Doubly Linked List Program (40 points)

Criteria	Points	Description
Correctness of find()	5	Correct implementation of the find() function, which returns the position of a given element in the list. The first element is at position 1.
Correctness of insertAfter()	5	Accurate implementation of the insertAfter() function (use the node returned by findNode() as the first parameter as an existing element). Inserts a new element (newElement) after a given element (existingElement). If the existingElement is not found, print a suitable message.
Correctness of reverse()	10	Accurate implementation of the reverse() function which modifies the list and reverses the order of list elements in place.
Correctness of addBack()	5	Accurate implementation of the addBack() function which adds a new element (newElement) at the back of the list.
Correctness of removeBack()	5	Correctness of removeBack() function which removes the element from the back of the list
Modification of main() for Testing	10	The original main() is modified correctly to test the new and existing functionalities without creating a new main() function.

2. Score Management Program (40 points)

Criteria	Points	Description
Correctness of findMinMaxScores()	10	Accurate implementation of the findMinMaxScores() function using std::pair<int, int> to return the minimum and maximum scores.
Correctness of averageMaxScores()	10	Correct implementation of the averageMaxScores() function, returning the correct average score for all entries.
Correctness of searchScore()	10	Accurate implementation of the searchScore() function to return the score for a specific player by name.
Modification of main() for Testing	10	The original main() is modified correctly to test the new and existing functionalities without creating a new main() function.

3. Submission Format (5 points)

Criteria	Points	Description
Correct Folder Structure	5	All files are correctly organized into folders named DLinkedProgram and ScoreManagementProgram. The submission folder is named as per instructions.

4. Screenshots (5 points)

Criteria	Points	Description
Correctness and Inclusion	5	Screenshots clearly showing program output are included in a separate MS Word document, covering all tested functions for both programs.

5. Thorough Testing (10 points)

Criteria	Points	Description
Test Coverage	10	Tests cover all key functions, including existing and find(), reverse(), insertAfter(), addBack(), removeBack() for Doubly Linked List, and findMinMaxScores(), averageMaxScores(), searchScore() for Score Management. Tests include edge cases (e.g., searching for an element not in the list).

Grading Summary:

- Doubly Linked List Program: 40 points
- Score Management Program: 40 points
- Correct Submission: 5 points
- Correct Screenshots: 5 points
- Thorough Testing: 10 points

End of Document

=====